

1. Scenario A — Basic Mixed Workload

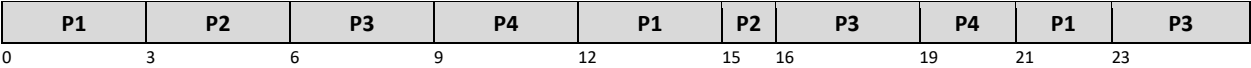
Processes: P1 (AT=0, BT=8), P2 (AT=1, BT=4), P3 (AT=2, BT=9), P4 (AT=3, BT=5)

Round Robin Quantum: Q = 3

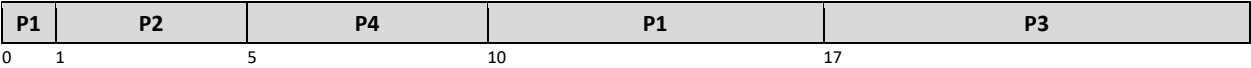
Purpose: A straightforward baseline — processes have different burst times and arrive at different times. This gives a clear initial picture of how each algorithm behaves on a normal mixed workload.

Gantt Charts

Round Robin (Q = 3):



SRTF:



Results

PID	AT	BT	RR WT	RR TAT	RR RT	SRTF WT	SRTF TAT	SRTF RT	Better WT
P1	0	8	15	23	0	9	17	0	SRTF
P2	1	4	11	15	2	0	4	0	SRTF
P3	2	9	15	24	4	15	24	15	Equal
P4	3	5	13	18	6	2	7	2	SRTF

AVERAGES			13.50	20.00	3.00	6.50	13.00	4.25	SRTF
----------	--	--	-------	-------	------	------	-------	------	------

2. Scenario B — Quantum Sensitivity

Processes: P1 (AT=0, BT=10), P2 (AT=0, BT=6), P3 (AT=0, BT=2), P4 (AT=0, BT=4)

All processes arrive at time 0 — no arrival-time variation to obscure the quantum effect.

Two quanta are tested: Q = 2 and Q = 5

Purpose: Show concretely how the choice of quantum changes Round Robin's behavior.

Scenario B1 — Small Quantum (Q = 2)

Round Robin (Q = 2):

P1	P2	P3	P4	P1	P2	P4	P1	P2	P1	P1
0	2	4	6	8	10	12	14	16	18	20

SRTF:

P3	P4	P2	P1
0	2	6	12

PID	AT	BT	RR WT	RR TAT	RR RT	SRTF WT	SRTF TAT	SRTF RT	Better WT
P1	0	10	12	22	0	12	22	12	Equal
P2	0	6	12	18	2	6	12	6	SRTF
P3	0	2	4	6	4	0	2	0	SRTF
P4	0	4	10	14	6	2	6	2	SRTF

AVERAGES	9.50	15.00	3.00	5.00	10.50	5.00	SRTF
----------	------	-------	------	------	-------	------	------

Scenario B2 — Large Quantum (Q = 5)

Round Robin (Q = 5):

P1	P2	P3	P4	P1	P2
0	5	10	12	16	21

SRTF results are the same as above (algorithm is independent of quantum).

PID	AT	BT	RR WT	RR TAT	RR RT	SRTF WT	SRTF TAT	SRTF RT	Better WT
P1	0	10	11	21	0	12	22	12	RR
P2	0	6	16	22	5	6	12	6	SRTF
P3	0	2	10	12	10	0	2	0	SRTF

PID	AT	BT	RR WT	RR TAT	RR RT	SRTF WT	SRTF TAT	SRTF RT	Better WT
P4	0	4	12	16	12	2	6	2	SRTF

AVERAGES	12.25	17.75	6.75	5.00	10.50	5.00	SRTF
----------	-------	-------	------	------	-------	------	------

Quantum Comparison (RR Only)

Metric	RR Q = 2	RR Q = 5	SRTF
Avg Waiting Time	9.50	12.25	5.00
Avg Turnaround	15.00	17.75	10.50
Avg Response Time	3.00	6.75	5.00

3. Scenario C — Short-Job Heavy Workload

Processes: P1 (AT=0, BT=2), P2 (AT=1, BT=1), P3 (AT=2, BT=3), P4 (AT=3, BT=1), P5 (AT=4, BT=2)

Round Robin Quantum: Q = 2

Purpose: Populate the workload with mostly short jobs to make SRTF's preference for short tasks visible.

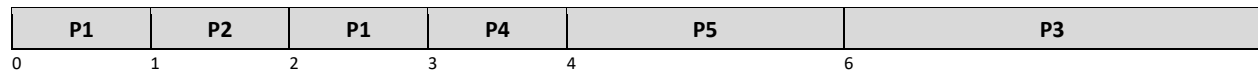
Four out of five processes have a burst of 1 or 2 — ideal candidates for SRTF preemption.

Gantt Charts

Round Robin (Q = 2):



SRTF:



Results

PID	AT	BT	RR WT	RR TAT	RR RT	SRTF WT	SRTF TAT	SRTF RT	Better WT
P1	0	2	0	2	0	1	3	0	RR
P2	1	1	1	2	1	0	1	0	SRTF
P3	2	3	4	7	1	4	7	4	Equal
P4	3	1	2	3	2	0	1	0	SRTF
P5	4	2	2	4	2	0	2	0	SRTF

AVERAGES	1.80	3.60	1.20	1.00	2.80	0.80	SRTF
----------	------	------	------	------	------	------	------

4. Scenario D — Interactive Fairness Case

Processes: P1 (AT=0, BT=6), P2 (AT=0, BT=6), P3 (AT=0, BT=6), P4 (AT=0, BT=6)

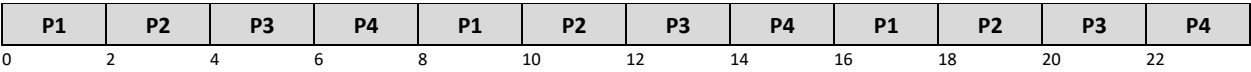
Round Robin Quantum: Q = 2

Purpose: All four processes are identical — same arrival time, same burst time.

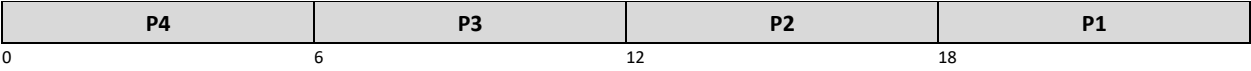
This removes all efficiency advantages and isolates fairness as the only variable.

Gantt Charts

Round Robin (Q = 2):



SRTF:



Results

PID	AT	BT	RR WT	RR TAT	RR RT	SRTF WT	SRTF TAT	SRTF RT	Better WT
P1	0	6	12	18	0	18	24	18	RR
P2	0	6	14	20	2	12	18	12	SRTF
P3	0	6	16	22	4	6	12	6	SRTF
P4	0	6	18	24	6	0	6	0	SRTF

AVERAGES			15.00	21.00	3.00	9.00	15.00	9.00	SRTF
----------	--	--	-------	-------	------	------	-------	------	------

5. Scenario E — Input Validation

Purpose: Confirm that the simulator rejects invalid inputs before any simulation runs.
Every invalid entry should produce a specific, clear error message. No partial run should occur.

Validation Test Cases

Input Field	Value Entered	Expected Response
Time Quantum (Q)	Q = 0	<i>Error: Quantum must be a positive integer (minimum 1).</i>
Time Quantum (Q)	Q = -3	<i>Error: Quantum must be a positive integer (minimum 1).</i>
Time Quantum (Q)	Q = 3.7 (decimal)	<i>Error: Quantum must be a whole number.</i>
Burst Time (BT)	BT = -5	<i>Error: Burst time must be greater than zero.</i>
Burst Time (BT)	BT = 0	<i>Error: Burst time must be greater than zero.</i>
Arrival Time (AT)	AT = -2	<i>Error: Arrival time cannot be negative.</i>
Process Count (N)	N = 0	<i>Error: At least one process must be defined.</i>

7. Analysis Questions

Which algorithm gave better average waiting time?

SRTF in Scenarios A, B, and C by a significant margin. Round Robin had lower average WT only in the tie conditions. Across realistic mixed workloads, SRTF consistently reduces total waiting.

Which algorithm gave better response time?

Round Robin in Scenarios A and D. Because RR cycles through all processes in every quantum window, each process gets its first turn quickly. SRTF can delay a long process's first response substantially if short jobs keep arriving.

Did Round Robin appear fairer across all processes?

Yes. The quantum bound ensures no process waits more than Q time units without getting a turn. In Scenario D, RR gave every process its first CPU slice within 6 time units. SRTF gave P1 a first response only after 18 time units — three times worse.

Did SRTF complete short jobs faster?

Yes, clearly. In Scenario C, both P2 (BT=1) and P4 (BT=1) had zero waiting time and zero response time under SRTF. Under RR, they waited 1 and 2 units respectively. SRTF's preemptive behavior gives short jobs immediate priority.

How did the selected quantum affect Round Robin results?

Scenario B makes this concrete. $Q=2$ produced average WT=9.50 and average RT=3.00. $Q=5$ produced average WT=12.25 and average RT=6.75. A larger quantum reduces context switching but hurts responsiveness significantly. The right quantum is the smallest value that keeps context-switch overhead manageable.

Which algorithm would you recommend for the tested workloads?

SRTF for batch or compute-heavy workloads where minimizing overall waiting and turnaround time is the priority. Round Robin for interactive or multi-user systems where every user expects a prompt first response and no process should dominate the CPU for long. The correct choice depends on the environment.

8. Conclusion

The five scenarios together give a reasonably complete picture of how these two schedulers behave. A few findings stood out clearly.

Performance on average waiting time. SRTF outperformed Round Robin in nearly every scenario. Its ability to immediately redirect the CPU to the shortest remaining job keeps the queue short and reduces the total time processes spend waiting. On average-WT alone, SRTF is the stronger algorithm.

Fairness. Round Robin is the fairer algorithm by design. The quantum guarantees that every process gets CPU time regularly, regardless of how long it has been running or how much work remains. Scenario D illustrated this most clearly: identical processes got response times of 0, 2, 4, and 6 under RR, compared to 18, 12, 6, and 0 under SRTF. If fairness matters — for example in a system serving multiple users simultaneously — Round Robin is the better fit.

SRTF and short jobs. SRTF benefits short jobs disproportionately. This is by design: the algorithm is built to favor processes that finish quickly. In Scenario C, nearly every short job received zero waiting time. The flip side is that long jobs can be delayed significantly if short jobs keep arriving — a condition known as starvation, which does not arise under Round Robin.

The quantum effect. Scenario B showed that the quantum is not a trivial setting. Moving from $Q=2$ to $Q=5$ worsened average waiting time by about 29% and nearly doubled average response time. A quantum that is too large pushes Round Robin toward FCFS behavior, eliminating most of its fairness benefits. Choosing a quantum that reflects the typical interaction time of the workload is important for getting good results from Round Robin.

Overall recommendation. Neither algorithm is universally better. SRTF maximizes throughput and minimizes average waiting time in batch-oriented settings. Round Robin protects every process's access to the CPU and delivers more predictable response times in interactive settings. Understanding the workload type and the goals of the system should drive the choice — and in many real operating systems, the answer is a combination of both ideas applied at different levels of the scheduler.
